7-Manipulating Data Objectives

Objective:
After completing this lesson, you should be able to do the following:
- **Describe each DML statement**
- **Insert rows into a table**
- **Update rows in a table**
- **Delete rows from a table**
- **Merge rows in a table**
- **Control transactions**

Data Manipulation Language

• A DML statement is executed when you:
–Add new rows to a table
–Modify existing rows in a table
–Remove existing rows from a table
•A transaction consists of a collection of DML statements that form a logical unit of work.

Adding a New Row to a Table

New row DEPARTMENTS
Insert a new row into the
DEPARMENTS
table.

The INSERT Statement Syntax

• Add new rows to a table by using the
INSERT statement.
INSERT INTO table[(
column[, column...])]
VALUES(value[, value...]);
•Only one row is inserted at a time with this syntax.
Adding a New Row to a Table (continued)
Note: This statement with the VALUES clause adds only one row at a time to a table.

Name	Null?	Type
DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

Inserting New Rows

• Insert a new row containing values for each column.
•List values in the default order of the columns in the table.
•Optionally, list the columns in the INSERT clause.

```
INSERT INTO departments(department_id, department_name,
manager_id, location_id)
VALUES      (70, 'Public Relations', 100, 1700);
1 row created.
```

•Enclose character and date values within single quotation marks.

Inserting Rows with Null Values

• Implicit method: Omit the column from the column list.

```
INSERT INTO departments (department_id,
department_name    )
VALUES (30, 'Purchasing');
1 row created.
```

•Explicit method: Specify the NULL keyword in the VALUES clause.

```
INSERT INTO departments
VALUES (100, 'Finance', NULL, NULL);
1 row created.
```

EMPLOYEE_ID	LAST_NAME	JOB_ID	HIRE_DATE	COMMISSION_PCT
113	Popp	AC_ACCOUNT	12-MAR-01	

Inserting Special Values

The SYSDATE function records the current date and time.

```
INSERT INTO employees (employee_id,
first_name, last_name,
email, phone_number,
hire_date, job_id, salary,
commission_pct, manager_id,
department_id)
VALUES (113, 'Louis', 'Popp', 'LPOPP', '515.124.4567',
SYSDATE, 'AC_ACCOUNT', 6900, NULL, 205, 100);
1 row created.
```

Confirming Additions to the Table

```
SELECT employee_id, last_name, job_id, hire_date, commission _pct
FROM   employees
WHERE  employee_id = 113;
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_P
114	Den	Raphealy	DRAPHEAL	515.127.4561	03-FEB-99	AC_ACCOUNT	11000	

Inserting Specific Date Values

•Add a new employee.

INSERT INTO employees
VALUES (114,
'Den', 'Raphealy',
'DRAPHEAL', '515.127.4561',
TO_DATE('FEB 3, 1999', 'MON DD, YYYY'),
'AC_ACCOUNT', 11000, NULL, 100, 30);
1 row created.

•Verify your addition.

INSERT INTO departments
(department_id, department_name, location_id)
VALUES (&department_id, '&department_name',&location);
40
Human Resources
2500
1 row created.

Copying Rows from Another Table

• Write your INSERT statement with a subquery.

INSERT INTO sales_reps(id, name, salary, commission_pct)
SELECT employee_id, last_name, salary, commission_pct
FROM employees
WHERE job_id LIKE '%REP%';
4 rows created.

•Do not use the VALUES clause.

• Match the number of columns in the INSERT clause to those in the subquery.

INSERT INTO copy_emp SELECT *
FROM employees;

The UPDATE Statement Syntax

•Modify existing rows with the
UPDATE
statement.

UPDATE ***table*** **SET** ***column = value*** **[,** ***column = value, ...*** **]**
[WHERE ***condition*** **];**

•Update more than one row at a time, if required.

Note: In general, use the primary key to identify a single row. Using other columns can unexpectedly cause several rows to be updated. For example, identifying a single row in the EMPLOYEES table by name is dangerous, because more than one employee may have the same name.

LAST_NAME	DEPARTMENT_ID
King	110
Kochhar	110
De Haan	110
Hunold	110
Ernst	110
Lorentz	110
Mourgos	110
Gietz	110

22 rows selected.

Updating Rows in a Table

•Specific row or rows are modified if you specify The WHERE clause.

```
UPDATE employees
SET    department_id = 70
WHERE  employee_id = 113;
1 row updated.
```

• All rows in the table are modified if you omit the WHERE clause.

```
UPDATE  copy_emp
SET     department_id = 110;
22 rows updated.
```

Updating Two Columns with a Subquery

Update employee 114's job and department to match that of employee 205.

```
UPDATE   employees
SET      job_id  = (SELECT  job_id
FROM     employees
WHERE    employee_id = 205),
salary  = (SELECT  salary
FROM     employees
WHERE    employee_id = 205)
WHERE    employee_id    =  114;
1 row updated.
```

Updating Rows Based on Another Table

Use subqueries in UPDATE statements to update rows in a table based on values from another table.

```
UPDATE  copy_emp
SET     department_id  =  (SELECT department_id
FROM employees
WHERE employee_id = 100)
WHERE   job_id        =  (SELECT job_id
FROM employees
WHERE employee_id = 200);
1 row updated.
```

Updating Rows:

Integrity Constraint Error

```
UPDATE employees
SET    department_id = 55
WHERE  department_id = 110;
UPDATE employees
*
ERROR at line 1:
ORA-02291: integrity constraint (HR.EMP_DEPT_FK)
violated - parent key not found
```

Integrity Constraint Error

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
70	Public Relations	100	1700
30	Purchasing		
50	Shipping	124	1500
60	IT	103	1400
100	Finance		
80	Sales	149	2500
DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
70	Public Relations	100	1700
30	Purchasing		
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500

Removing a Row from a Table

DEPARTMENTS

Delete a row from the DEPARTMENTS table.

The DELETE Statement

You can remove existing rows from a table by using
The DELETE statement.

DELETE [FROM] table
[WHERE condition];

Note: If no rows are deleted, a message "
0 rows deleted ." is returned:

Deleting Rows from a Table

•Specific rows are deleted if you specify the
WHERE clause.

DELETE FROM departments
WHERE department_name = 'Finance';
1 row deleted.

•All rows in the table are deleted if you omit the
WHERE clause**.**

DELETE FROM copy_emp;
22 rows deleted.

Example

Remove rows identified in the WHERE clause.

```
DELETE FROM  employees
WHERE       employee_id = 114;
1 row deleted.
DELETE FROM  departments
WHERE       department_id IN (30, 40);
2 rows deleted.
```

Deleting Rows Based on Another Table

Use subqueries in DELETE
statements to remove
rows from a table based on values from another table.

DELETE FROM employees
WHERE department_id =
(SELECT department_id
FROM departments
WHERE department_name LIKE '%Public%');
1 row deleted.

Deleting Rows:

Integrity Constraint Error

DELETE FROM departments
WHERE department_id = 60;
DELETE FROM departments
*****
ERROR at line 1:

ORA-02292: integrity constraint (HR.EMP_DEPT_FK)
violated - child record found
Integrity Constraint Error
The following statement works because there are no employees in department 70:
DELETE FROM departments WHERE department_id = 70;
1 row deleted.

Using a Subquery in an INSERT Statement

INSERT INTO
(SELECT employee_id, last_name,
email, hire_date, job_id, salary,
department_id
FROM employees
WHERE department_id = 50)
VALUES (99999, 'Taylor', 'DTAYLOR',
TO_DATE('07-JUN-99', 'DD-MON-RR'),
'ST_CLERK', 5000, 50);

1 row created.

EMPLOYEE_ID	LAST_NAME	EMAIL	HIRE_DATE	JOB_ID	SALARY	DEPARTMENT_ID
124	Mourgos	KMOURGOS	16-NOV-99	ST_MAN	5800	50
141	Rajs	TRAJS	17-OCT-95	ST_CLERK	3500	50
142	Davies	CDAVIES	29-JAN-97	ST_CLERK	3100	50
143	Matos	RMATOS	15-MAR-98	ST_CLERK	2600	50
144	Vargas	PVARGAS	09-JUL-98	ST_CLERK	2500	50
99999	Taylor	DTAYLOR	07-JUN-99	ST_CLERK	5000	50

6 rows selected

Using a Subquery in an INSERT Statement

•Verify the results

SELECT employee_id, last_name, email, hire_date,
job_id, salary, department_id
FROM employees
WHERE department_id = 50;

Using the WITH CHECK OPTION Keyword on DML Statements

•A subquery is used to identify the table and columns of the DML statement.
•The WITH CHECK OPTION keyword prohibits you from changing rows that are not in the subquery.

INSERT INTO (SELECT employee_id, last_name, email,
hire_date, job_id, salary
FROM employees

```
WHERE  department_id = 50 WITH CHECK OPTION)
VALUES (99998, 'Smith', 'JSMITH',
TO_DATE('07-JUN-99', 'DD-MON-RR'),
'ST_CLERK', 5000);
INSERT INTO
*
ERROR at line 1:
ORA-01402: view WITH CHECK OPTION where-clause violation
```

The MERGE Statement

•Provides the ability to conditionally update or insert data into a database table
•Performs an
UPDATE
if the row exists and an
INSERT
if it is a new row:
–Avoids separate updates
–Increases performance and ease of use
–Is useful in data warehousing applications

MERGE Statement Syntax

You can conditionally insert or update rows in a table by using the MERGE statement.

```
MERGE INTO table_name
AS table_alias
USING ( table|view|sub_query) AS alias
ON (join condition)
WHEN MATCHED THEN
UPDATE SET
col1 = col_val1,
col2 = col2_val
WHEN NOT MATCHED THEN
INSERT (column_list)
VALUES (column_values);
```

Merging Rows

Insert or update rows in the OPY_EMP table to match the EMPLOYEES table.

```
MERGE INTO copy_emp AS c
USING employees e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
```

```
c.first_name     = e.first_name,
c.last_name      = e.last_name,
...
c.department_id  = e.department_id
WHEN NOT MATCHED THEN
INSERT VALUES(e.employee_id, e.first_name, e.last_name,
e.email, e.phone_number, e.hire_date, e.job_id,
e.salary, e.commission_pct, e.manager_id,
e.department_id);
```

Merging Rows: Example

```
MERGE INTO copy_emp AS c
USING employees e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
c.first_name     = e.first_name,
c.last_name      = e.last_name,
c.email          = e.email,
c.phone_number   = e.phone_number,
c.hire_date      = e.hire_date,
c.job_id         = e.job_id,
c.salary         = e.salary,
c.commission_pct = e.commission_pct,
c.manager_id     = e.manager_id,
c.department_id  = e.department_id
WHEN NOT MATCHED THEN
INSERT VALUES(e.employee_id, e.first_name, e.last_name,
e.email, e.phone_number, e.hire_date, e.job_id,
e.salary, e.commission_pct, e.manager_id,
e.department_id);
```

Merging Rows

```
SELECT *
FROM COPY_EMP;
no rows selected
MERGE INTO copy_emp c
USING employees e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
...
WHEN NOT MATCHED THEN
INSERT VALUES...;
SELECT *
FROM COPY_EMP;
20 rows selected.
```

Database Transactions

A database transaction consists of one of the following:
•DML statements which constitute one consistent change to the data
•One DDL statement
•One DCL statement

Database Transactions

•Begin when the first DML SQL statement is executed
•End with one of the following events:
–A COMMIT or ROLLBACK statement is issued
–A DDL or DCL statement executes (automatic commit)
–The user exits iSQL*Plus
–The system crashes

Advantages of COMMIT and ROLLBACK Statements With COMMIT and ROLLBACK

statements, you can:
•Ensure data consistency
•Preview data changes before making changes permanent
•Group logically related operations
Controlling Transactions
COMMIT Time
Transaction
DELETE
SAVEPOINT A
INSERT
UPDATE
SAVEPOINT B
INSERT
ROLLBACK
ROLLBACK
ROLLBACK
to SAVEPOINT A
to SAVEPOINT B
Note:SAVEPOINT is not ANSI standard SQL.

Rolling Back Changes to a Marker

• Create a marker in a current transaction by using The SAVEPOINT statement.
•Roll back to that marker by using the ROLLBACK TO SAVEPOINT statement.

UPDATE... SAVEPOINT update_done;
Savepoint created.
INSERT...
ROLLBACK TO update_done;
Rollback complete.

Implicit Transaction Processing

• An automatic commit occurs under the following circumstances:
–DDL statement is issued
–DCL statement is issued
–Normal exit from iSQL*Plus, without explicitly Issuing COMMIT or ROLLBACK statements
•An automatic rollback occurs under an abnormal termination of iSQL*Plus or a system failure.
Implicit Transaction Processing
Status Circumstances

State of the Data Before COMMIT Or ROLLBACK

•The previous state of the data can be recovered.
•The current user can review the results of the DML operations by using the SELECT statement.
•Other users cannot view the results of the DML statements by the current user.
• The affected rows are locked; other users cannot change the data within the affected rows.

State of the Data After COMMIT

•Data changes are made permanent in the database.
•The previous state of the data is permanently lost.
•All users can view the results.
•Locks on the affected rows are released; those rows are available for other users to manipulate.
•All savepoints are erased.

Committing Data

•Make the changes.
DELETE FROM employees
WHERE employee_id = 99999;
1 row deleted.
INSERT INTO departments
VALUES (290, 'Corporate Tax', NULL, 1700);
1 row inserted.
•Commit the changes.
COMMIT;

Commit complete.
Example
Remove departments 290 and 300 in the DEPARTMENTS table, and update a row in the COPY_EMP table. Make the data change permanent.

```
DELETE FROM departments
WHERE  department_id IN (290, 300);
2 rows deleted.
UPDATE copy_emp
SET department_id = 80
WHERE employee_id = 206;
1 row updated.
COMMIT;
Commit Complete.
```

State of the Data After ROLLBACK

Discard all pending changes by using the
ROLLBACK
statement:
•Data changes are undone.
•Previous state of the data is restored.
•Locks on the affected rows are released.

```
DELETE FROM copy_emp;
22 rows deleted.
ROLLBACK;
Rollback complete.
```

Rolling Back Changes

```
DELETE FROM  test;
25,000 rows deleted.
ROLLBACK;
Rollback complete.
DELETE FROM  test
WHERE       id = 100;
1 row deleted.
SELECT   *
FROM     test
WHERE    id = 100;
No rows selected.
COMMIT;
Commit complete.
```

Statement-Level Rollback

•If a single DML statement fails during execution, only that statement is rolled back.
• The Oracle Server implements an implicit savepoint.
•All other changes are retained.
•The user should terminate transactions explicitly

by executing a COMMIT or ROLLBACK
statement.

Read Consistency

•Read consistency guarantees a consistent view of the data at all times.
•Changes made by one user do not conflict with changes made by another user.
•Read consistency ensures that on the same data:
–Readers do not wait for writers
–Writers do not wait for readers

Implementation of Read Consistency

User A Data
UPDATE employees blocks
SET salary = 7000
WHERE last_name = 'Goyal';
Undo segments changed and
SELECT *
Read unchanged
FROM userA.employees;
Data consistent before image change: “old” data
User B

Locking

In an Oracle database, locks:
•Prevent destructive interaction between concurrent transactions
•Require no user action
•Use the lowest level of restrictiveness
•Are held for the duration of the transaction
•Are of two types: explicit locking and implicit locking

What Are Locks?

Locks are mechanisms that prevent destructive interaction between tra nsactions accessing the same resource, either a user object (such as tables or rows) or a system object not visible to users (such as shared data structures and data dictionary rows).

How the Oracle Database Locks Data

Locking is performed automatically and requires no user action. Implicit locking occurs for SQL statements as necessary, depending on the action requested. Implicit locking occurs for all SQL statements except SELECT.
The users can also lock data manually, which is called explicit locking.

Implicit Locking

•Two lock modes:

–Exclusive: Locks out other users
–Share: Allows other users to access the server
•High level of data concurrency:
–DML: Table share, row exclusive
–Queries: No locks required
–DDL: Protects object definitions
•Locks held until commit or rollback

DML Locking

When performing data manipulation language (DML) operations, the Oracle Server provides data concurrency through DML locking. DML locks occur at two levels:
• A share lock is automatically obtained at the table level during DML operations. With share lock mode, several transactions can acquire share locks on the same resource.
• An exclusive lock is acquired automatically for each row modified by a DML statement. Exclusive locks prevent the row from being changed by other transactions until the transaction is committed or rolled back. This lock ensures that no other user can modify the same row at the same time and
overwrite changes not yet committed by another user.
Note: DDL locks occur when you modify a database object such as a table.

Practice 7

Insert data into the MY_EMPLOYEE table.
1. Run the statement in the lab8_1.sql script to build the MY_EMPLOYEE table to be used for the lab.
2. Describe the structure of the MY_EMPLOYEE table to identify the column names.
3. Add the first row of data to the MY_EMPLOYEE table from the following sample data. Do not list the columns in the INSERT clause.

ID LAST_NAME FIRST_NAME USERID SALARY
1 Patel Ralph rpatel 895
2 Dancs Betty bdancs 860
3 Biri Ben bbiri 1100
4 Newman Chad cnewman 750
5 Ropeburn Audrey aropebur 1550

4. Populate the MY_EMPLOYEE table with the second row of sample data from the preceding list. This time, list the columns explicitly in the INSERT clause.
5. Confirm your addition to the table.

6. Write an INSERT statement in a text file named loademp.sql to load rows into the MY_EMPLOYEE table. Concatenate the first letter of the first name and the f irst seven characters of the last name to produce the user ID.
7. Populate the table with the next two rows of sample data by runn ing the INSERT statement in the script that you created.
8. Confirm your additions to the table.
9. Make the data additions permanent.

Update and delete data in the MY_EMPLOYEE table.
10. Change the last name of employee 3 to Drexler.
11. Change the salary to 1000 for all employees with a salary less than 900.
12. Verify your changes to the table.
13. Delete Betty Dancs from the MY_EMPLOYEE table.
14. Confirm your changes to the table.
15. Commit all pending changes. Control data transaction to the MY_EMPLOYEE table.
16. Populate the table with the last row of sample data by modifying the statements in the script that you created in step 6. Run the statements in the script.
17. Confirm your addition to the table.
18. Mark an intermediate point in the processing of the transaction.
19. Empty the entire table.
20. Confirm that the table is empty.
21. Discard the most recent DELETE operation without discarding the earlier INSERT operation.
22. Confirm that the new row is still intact.
23. Make the data addition permanent.